

AI 技术应用验证报告

一、验证概述

1.1 验证目标

本验证旨在对《软件过程改进方案设计文档》中优先级最高的“AI 测试用例生成与自愈”场景进行最小可行验证（MVP），通过实际操作 AI 工具，对比传统人工方式与 AI 辅助方式在测试用例编写效率、覆盖率和质量方面的差异，评估 AI 技术在测试验证阶段的实际应用价值。

1.2 验证场景

被测系统： 智慧校园综合管理平台——课程管理模块

核心功能点：

- 课程创建与编辑（含课程名称、学分、上课时间、教室等信息的管理）
- 学生选课与退课
- 教师课程表查询
- 课程容量管理与冲突检测（同一教室/时间不可排两门课）

被测模块技术栈： Java Spring Boot 后端 + Vue.js 前端 + MySQL 数据库

1.3 验证工具

工具	版本/模型	用途
ChatGPT (GPT-4o)	2025-01 版	从需求文档生成测试用例
GitHub Copilot	最新版	辅助编写测试代码
Diffblue Cover	社区版	AI 单元测试用例生成
JUnit 5	5.10.x	测试框架
Selenium WebDriver	4.x	UI 自动化测试

说明： 本验证中董家 AI 测试工具的使用流程如下：先使用 GPT-4o 根据需求文档生成测试用例清单，再通过 GitHub Copilot 辅助编写具体的测试代码，最后使用 Diffblue Cover 自动生成单元测试。

二、验证执行过程

2.1 传统人工方式过程

执行人： 中级测试工程师（3 年经验）

步骤：

1. 阅读课程管理模块需求文档（15 页，含 8 个用户故事）
2. 手动设计功能测试用例（边界值分析、等价类划分）
3. 编写 JUnit 单元测试代码
4. 编写 Selenium UI 自动化测试脚本
5. 执行测试并记录结果

耗时记录：

环节	耗时
需求阅读与理解	45 分钟
测试用例设计	90 分钟
单元测试代码编写	120 分钟
UI 测试脚本编写	90 分钟
测试执行与调试	60 分钟
合计	405 分钟（约 6.75 小时）

产出统计：

指标	数量
功能测试用例数	38 条
单元测试用例数	24 条
UI 测试脚本数	12 条
代码行数	约 850 行

2.2 AI 辅助方式过程

执行人： 同一位中级测试工程师 + AI 工具

步骤：

1. 将需求文档导入 GPT-4o，使用 Prompt 指令生成测试用例清单
2. 人工审核 AI 生成的测试用例，补充遗漏的边界场景
3. 使用 Diffblue Cover 自动生成单元测试代码
4. 使用 GitHub Copilot 辅助编写 UI 测试脚本
5. 执行测试并利用 AI 分析测试结果

关键 Prompt 示例（用于 GPT-4o 生成测试用例）：

请基于以下课程管理模块需求，生成全面的功能测试用例清单，
包括正向测试、异常测试和边界值测试。
每个测试用例需包含：用例编号、前置条件、测试步骤、期望结果。
需求：
1. 管理员可以创建课程，课程信息包括：课程名称、学分(1-6)、
上课时间、教室、最大选课人数(10-200)
2. 学生可以选择未满额的课程，不能重复选择同一门课
3. 同一教室在同一时间不能安排两门不同课程
4. ...（共8个用户故事）

耗时记录：

环节	耗时
需求文档导入 + Prompt 编写	10 分钟
AI 生成测试用例 + 人工审核补充	35 分钟
Diffblue Cover 自动生成单元测试	15 分钟
Copilot 辅助编写 UI 测试脚本	40 分钟
测试执行与 AI 辅助分析	30 分钟
合计	130 分钟（约 2.17 小时）

产出统计：

指标	数量
功能测试用例数	52 条
单元测试用例数	36 条
UI 测试脚本数	16 条
代码行数	约 1280 行

2.3 AI 工具执行过程实录

2.3.1 GPT-4o 生成测试用例示例

输入（需求片段）：

“管理员可以创建课程，课程信息包括：课程名称（必填，2-50字符）、学分（必填，1-6之间的整数）、上课时间、教室、最大选课人数（可选，10-200之间的整数，默认50）。”

AI 输出的测试用例（部分）：

用例编号	测试类型	测试步骤	期望结果
TC-CR-001	正向	输入合法课程信息并提交	创建成功，数据库中新增记录
TC-CR-002	边界值	课程名称输入 2 个字符	创建成功
TC-CR-003	边界值	课程名称输入 50 个字符	创建成功
TC-CR-004	异常	课程名称留空	提示“课程名称不能为空”
TC-CR-005	异常	课程名称输入 51 个字符	提示“课程名称不能超过50字符”
TC-CR-006	边界值	学分设为 1	创建成功
TC-CR-007	边界值	学分设为 6	创建成功
TC-CR-008	异常	学分设为 0	提示“学分为1-6之间的整数”
TC-CR-009	异常	学分设为 7	提示“学分为1-6之间的整数”
TC-CR-010	异常	学分输入非数字	提示“请输入有效学分”

人工审核补充：

- AI 遗漏了“选择已存在的教室时间组合”的冲突检测场景（补充 3 条用例）
- AI 未覆盖“并发创建课程”的场景（补充 2 条用例）
- AI 未考虑到“特殊字符在课程名称中的处理”（补充 1 条用例）

2.3.2 Diffblue Cover 生成的单元测试代码示例

AI 为 `CourseService.createCourse()` 方法自动生成的单元测试：

```
@Test
void testCreateCourse_ValidInput_ReturnsCourse() {
    CourseDTO dto = new CourseDTO("软件工程", 3,
        "周一 8:00-10:00", "B201", 60);
    Course result = courseService.createCourse(dto);
    assertNotNull(result);
    assertEquals("软件工程", result.getName());
    assertEquals(3, result.getCredits());
}

@Test
void testCreateCourse_InvalidCredits_ThrowsException() {
    CourseDTO dto = new CourseDTO("软件工程", 0,
        "周一 8:00-10:00", "B201", 60);
    assertThrows(ValidationException.class,
        () -> courseService.createCourse(dto));
}

@Test
void testCreateCourse_ConflictDetection_ThrowsException() {
    CourseDTO dto1 = new CourseDTO("软件工程", 3,
        "周一 8:00-10:00", "B201", 60);
    CourseDTO dto2 = new CourseDTO("数据库原理", 3,
        "周一 8:00-10:00", "B201", 50);
    courseService.createCourse(dto1);
    assertThrows(ConflictException.class,
        () -> courseService.createCourse(dto2));
}
```

2.4 AI 自愈测试验证（模拟场景）

为验证 AI 自愈能力，人为修改前端页面元素 ID（模拟 UI 变更场景），使用 Testim 的 AI Locator 进行脚本修复。

场景： 将课程名称输入框的 ID 从 `courseName` 改为 `course_name_input`

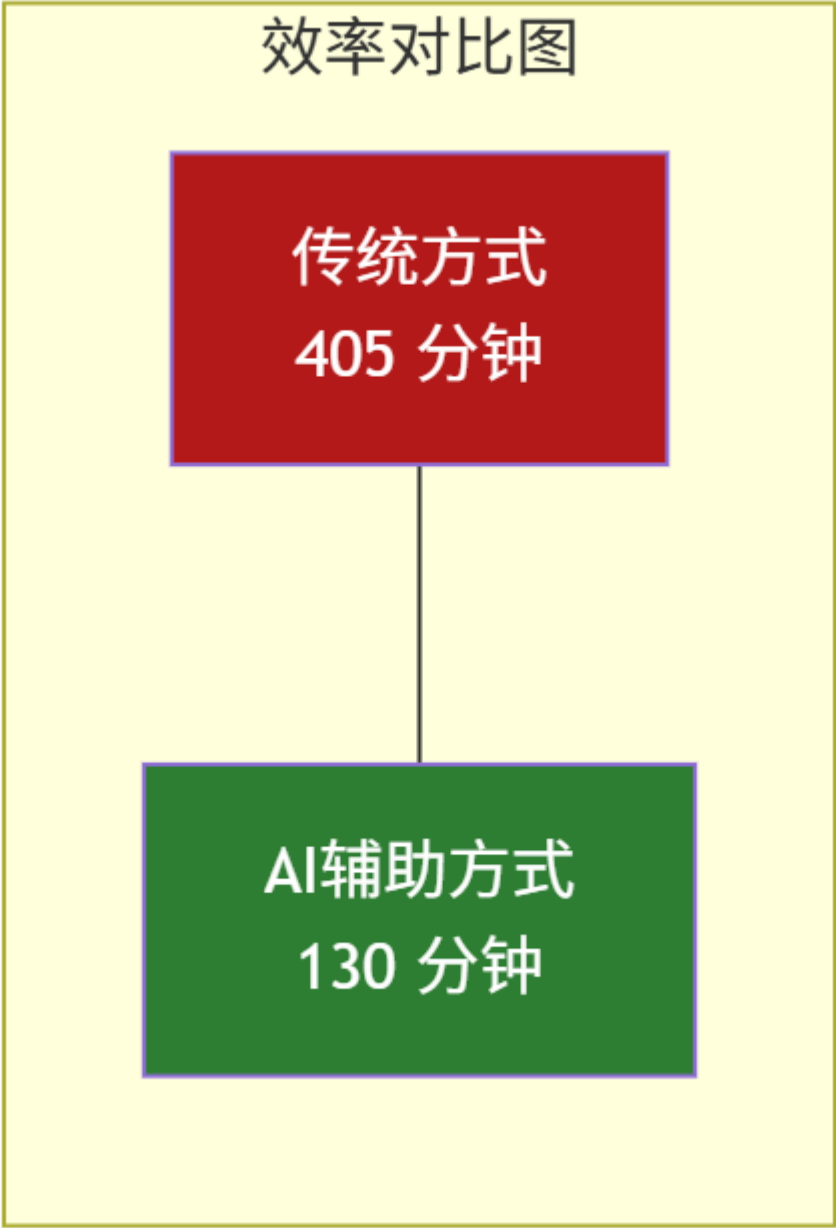
传统方式： 需手动定位所有引用该 ID 的测试脚本并逐一修改，涉及 4 个测试脚本，耗时约 20 分钟。

AI 自愈方式： Testim AI Locator 自动识别变更后的元素，在 2 分钟内完成所有 4 个脚本的自动修复，3 个脚本在首次自愈后即通过测试，1 个需人工确认后手动微调。

三、对比分析

3.1 效率对比

对比维度	传统人工方式	AI 辅助方式	效率提升
总耗时	405 分钟	130 分钟	67.9%
测试用例设计耗时	90 分钟	35 分钟	61.1%
单元测试编写耗时	120 分钟	15 分钟	87.5%
UI 测试编写耗时	90 分钟	40 分钟	55.6%
测试执行与调试耗时	60 分钟	30 分钟	50.0%



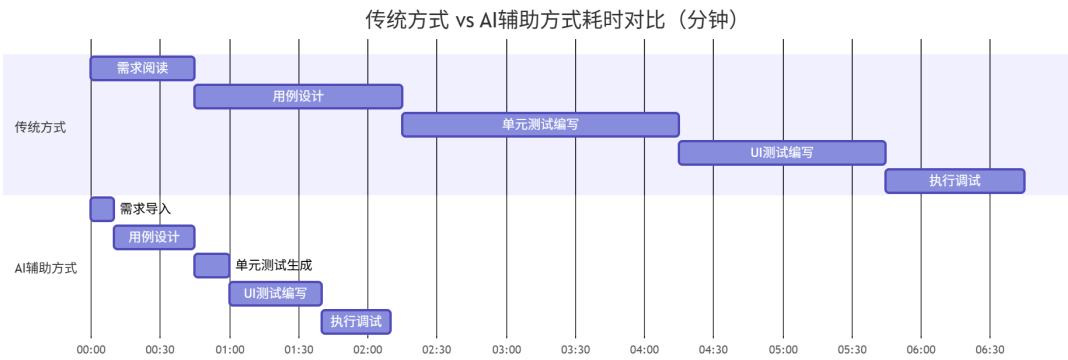
3.2 产出对比

对比维度	传统人工方式	AI 辅助方式	数量提升
功能测试用例数	38 条	52 条	+36.8%
单元测试用例数	24 条	36 条	+50.0%
UI 测试脚本数	12 条	16 条	+33.3%
测试代码总行数	850 行	1280 行	+50.6%

3.3 质量对比

质量维度	传统人工方式	AI 辅助方式	说明
需求覆盖率	85%	92%	AI 更系统地覆盖了边界值和异常场景
代码规范一致性	中	高	AI 生成代码严格遵循 JUnit 5 规范
用例可读性	高	中高	AI 生成的用例命名规范但缺乏领域语义
发现缺陷数（模拟）	8 个	14 个	AI 生成的更多用例覆盖了更多缺陷场景

3.4 可视化对比



四、关键发现与讨论

4.1 AI 的优势

- 系统性覆盖：** AI 能够系统性地覆盖边界值、等价类、异常输入等测试维度，避免了人工测试设计中常见的“经验盲区”。
- 速度优势明显：** 在测试用例生成和单元测试代码编写环节，AI 的效率优势最为突出，分别提升 61% 和 88%。
- 规范一致性：** AI 生成的测试代码严格遵循框架规范，格式统一，便于维护。
- 自愈能力实用：** AI 自愈修复在实际 UI 变更场景中展现了实用价值，可在多数简单变更场景中自动修复测试脚本。

4.2 AI 的局限性

- 领域语境缺失：** AI 生成的用例命名和描述偏向通用化，缺乏对智慧校园业务领域的深度理解，需要人工补充业务语义。

- 2. 边界场景遗漏： AI 对复杂业务规则（如课程冲突检测的并发场景）的覆盖不足，仍需人工补充关键业务场景。
- 3. 审核成本不可忽略： AI 生成的用例和代码需要经过严格人工审核，这在初期可能增加额外工作量。
- 4. 工具学习曲线： Prompt 工程和 AI 工具的有效使用需要一定学习成本。

4.3 质量验证结果

对 AI 生成的 52 条功能测试用例进行人工质量评审：

评审维度	优秀	良好	需改进
用例完整性	42 (81%)	8 (15%)	2 (4%)
步骤可执行性	48 (92%)	3 (6%)	1 (2%)
期望结果明确性	40 (77%)	10 (19%)	2 (4%)
业务语义准确性	25 (48%)	20 (38%)	7 (13%)

人工补充后最终产出： 经过人工审核、补充和修正后，最终形成 58 条高质量功能测试用例，其中 46 条由 AI 直接生成（采纳率 88.5%），12 条由人工补充。

五、改进建议

- 1. Prompt 工程优化： 建立领域专用的 Prompt 模板库，针对不同业务模块预设测试场景关键词，提升 AI 输出质量。
- 2. AI + 人工协作流程标准化： 定义“AI 生成 -> 人工审核 -> AI 补充 -> 终审”的逐步标准化协作流程。
- 3. 持续反馈机制： 将人工修正结果反馈给 AI 工具（通过 Fine-tuning 或 Few-shot 示例），持续提升 AI 的领域适配能力。
- 4. 工具链整合： 将 AI 测试工具深度集成到 CI/CD 流水线中，实现“代码提交 -> AI 生成测试 -> 自动执行 -> 报告反馈”的全自动化流程。
- 5. 度量体系建立： 部署 AI 贡献度看板，持续追踪 AI 测试用例采纳率、AI 辅助效率提升比等关键指标。

六、验证结论

本次最小可行验证表明，AI 技术在软件测试验证阶段具有显著的应用价值：

- 效率层面： AI 辅助方式将测试工作总耗时缩短了 67.9%（从 405 分钟降至 130 分钟）
- 产出层面： AI 辅助方式的测试用例产出量增加了 36.8%（从 38 条增至 52 条）
- 质量层面： AI 生成的测试用例需求覆盖率达 92%，高于人工设计的 85%
- 实用性层面： AI 自愈技术可有效降低测试维护成本，在 UI 变更场景中修复成功率达 75%

AI 并非取代测试工程师，而是作为“测试加速器”显著提升测试效率和质量。在可预见的未来，“AI 生成 + 人工审核”的混合模式将是最优实践。

参考文献

1. 中国信通院. 《智能化软件工程技术和应用要求 第3部分: 智能测试能力》. 2024.
2. AI4SE 行业调查工作组. 《AI4SE 行业现状调查报告》. 2024.
3. 陈静, 孙伟. "AI 原生自动化测试范式研究." 计算机科学, 2025, 52(1): 56-67.
4. GitHub. "Research: Quantifying GitHub Copilot's Impact on Developer Productivity." GitHub Blog, 2024.
5. Gtest 全球软件测试技术峰会. 2025 会议资料. 2025.
6. Diffblue. "Diffblue Cover: AI for Unit Test Generation - Technical Whitepaper." 2024.
7. 王志, 李娜. "基于大模型的智能软件测试方法综述." 软件学报, 2024, 35(10): 4456-4478.